Based on the Hierarchy (Basic Information)

Collection<E> : Predefined interface available from JDK 1.2V, It represents the root hierarchy for all the collection classes & interfaces.

Iterable<E> : Predefined interface available from JDK 1.5V in java.lang package. It provides all the iterable features and for Collection classes but for entire java.

Iterator<E> : Predefined interface available from JDK 1.2V, It reads the collection object in forward direction.

ListIterator<E> : Predefined interface available from JDK 1.2V, It reads the collection object in **forward + backward** both the direction. It is the sub interface of Iterator<E> and not to based on the index.

List<E> : Predefined interface available from JDK 1.2V, It accepts duplicate elements.

Set<E> : Predefined interface available from JDK 1.2V, It does not accept duplicate elements. (hashCode() + equals())

Queue<E> : Predefined interface available from JDK 1.5V. It works based on the FIFO (Fisrt In First Out)

Collections : It is a predefined utility class available from JDK 1.2V in java.util package, It provides various useful methods to work with Collection Objects.

Methods of Collection interface :



a) public boolean add(E element) : It is used to add an item/element in the in the collection.

Example :
```
Collection v = new Vector();
v.add(111);
v.add(112);
v.add(113);
```

b) public boolean addAll(Collection c) : It is used to insert the specified collection elements in the existing collection(The merging the Collection)

Example :
```
Vector<Integer> v1 = new Vector<>();
        v1.add(100);
        v1.add(200);

        Vector<Integer> v2 = new Vector<>();
        v2.add(300);
        v2.add(400);

        v1.addAll(v2);
        IO.println(v1);
```

c) public boolean retainAll(Collection c) : It is used to retain all the elements from existing element. (Common Element)

Example :
```
Vector<Integer> v1 = new Vector<>();
        v1.add(100);
        v1.add(200);

        Vector<Integer> v2 = new Vector<>();
        v2.add(100);
        v2.add(200);

        v1.retainAll(v2);
        IO.println(v1);
```

d) public boolean removeAll(COLLECTION c) : It is used to delete all the elements from the existing collection.

Example :
```
Vector<Integer> v1 = new Vector<>();
        v1.add(100);
        v1.add(200);
        v1.add(300);
        v1.add(400);

        Vector<Integer> v2 = new Vector<>();
        v2.add(100);
        v2.add(200);
        v2.add(300);
```

e) public boolean remove(Object element) : It is used to delete an element from the collection based on the object.

Example :
```
Vector<Integer> v1 = new Vector<>();
        v1.add(100);
        v1.add(200);
        v1.add(300);


        v1.remove(Integer.valueOf(100));
```

f) public int size() : It is used to find out the size of the Collection (Total number of elements available)

Example :
```
Vector<Integer> v1 = new Vector<>();
        v1.add(100);
        v1.add(200);
        v1.add(300);


        IO.println(v1.size());
```

g) public void clear() : It is used to clear all the elements at once from the Collection.

Example :
```
Vector<Integer> v1 = new Vector<>();
        v1.add(100);
        v1.add(200);
        v1.add(300);

        v1.clear();

        IO.println(v1); //[]
```

h) default boolean removeIf(Predicate<? p) : It will remove the elements from the Collection based on boolean expression using Predicate.

Example :
```
Vector<Integer> v1 = new Vector<>();
        v1.add(1);
        v1.add(2);
        v1.add(3);
        v1.add(4);
        v1.add(5);
        v1.add(6);
        int num = 3;
        v1.removeIf(n -> n>num-1);

        IO.println(v1);
```

i) public boolean contains(Object obj) : Used to search an Object in the given collection.

Example
```
Vector<Integer> v1 = new Vector<>();
        v1.add(111);
        v1.add(112);
        v1.add(113);

        IO.println(v1.contains(111));
```

j) public Object[] toArray() : Used to convert the given Collection into array object.

Example
```
ArrayList<Integer> v1 = new ArrayList<>();
        v1.add(111);
        v1.add(112);
        v1.add(113);

        Object[] array = v1.toArray();

        IO.println(Arrays.toString(array));
```

All the above methods of Collection interface will be applicable to all the sub interfaces like List, Set and Queue.